

AI-Assisted Formalization of Sharpness for Bernoulli Bond Percolation in Lean

Process note

June 2026

Abstract

We describe a Lean formalization of a self-contained proof of sharpness for nearest-neighbor Bernoulli bond percolation on $\mathbb{Z}^d$. The formalized mathematical target is the standard sharpness package: exponential decay of finite-distance connection probabilities below the critical point and a quantitative lower bound on the infinite-cluster probability above the critical point. The proof follows the finite-set criterion of Duminil-Copin and Tassion, specialized to nearest-neighbor bond percolation. The project was carried out with extensive use of an AI-assisted workflow: first producing a gap-filled proof note, then converting it into a Lean-oriented blueprint, and finally running a Codex CLI agent through a sequence of verified milestones. The main methodological lesson is that the AI system was most effective when the global proof architecture had already been fixed: it first produced the framework and routine proofs, and then used repeated Lean feedback loops to close local formalization gaps.

Key words and phrases. Bernoulli percolation, sharpness, phase transition, Lean, formalized mathematics, AI-assisted theorem proving.

2020 Mathematics Subject Classification. 60K35, 68V20, 82B43.

Contents

1	Introduction	2
2	The formalized percolation theorem	3
2.1	Percolation basics	3
2.2	The finite-set criterion	4
2.3	Technical lemmas	4
3	Lean file structure	5
3.1	Foundational files	6
3.2	Proof-spine files	6
3.3	Dependency graph	7
3.4	Milestone structure	7
4	AI-assisted workflow	8
4.1	Step 1: make the proof formalization-ready	8
4.2	Step 2: ask for a Lean blueprint	8
4.3	Step 3: generate a Codex CLI agent	9
4.4	Step 4: run, check, and repair	9
4.5	What made the workflow robust	10

1 Introduction

Bernoulli bond percolation is one of the basic models of probability theory and statistical mechanics. On the nearest-neighbor graph of $\mathbb{Z}^d$, each edge is declared open independently with probability p . The central object is the phase transition between the subcritical regime, in which long open connections are rare, and the supercritical regime, in which the origin has positive probability to belong to an infinite open cluster. Writing

$$\theta(p) := \mathbb{P}_p(0 \longleftrightarrow \infty), \quad p_c := \inf\{p \in [0, 1] : \theta(p) > 0\},$$

the sharpness theorem asserts that this transition is not merely qualitative: below p_c connection probabilities decay exponentially, while above p_c the order parameter is bounded from below linearly in $p - p_c$ up to the natural normalization.

The proof formalized in this project is a self-contained version of the Duminil-Copin–Tassion sharpness argument [2], restricted to nearest-neighbor bond percolation on $\mathbb{Z}^d$. This restriction is mathematically modest but formalization-friendly. The nearest-neighbor geometry makes boundary edges, finite boxes, last-exit arguments, and translation estimates explicit. The proof also avoids the susceptibility estimate, since in this setting the boundary inequality already yields the desired distance-to-boundary exponential decay.

The formalization is interesting for two reasons. First, the theorem is a genuine theorem of statistical mechanics and probability, involving an infinite graph, product measures, connectivity events, independence, differentiation of event probabilities, and a limiting critical point. Second, the proof is short on paper but contains many hidden formal obligations. One must show, for example, that apparently infinite connection events depend on finitely many edges when a finite target set is used; that pivotal decompositions preserve the correct orientations of boundary edges; that independence claims really follow from disjoint finite supports; and that the endpoint cases in the comparison of critical points are not silently ignored.

The workflow was modeled on recent formalization papers in which the mathematical result is stated first, followed by the proof architecture and the organization of the Lean development. In particular, Armstrong and Kempe’s paper on the Lean formalization of De Giorgi–Nash–Moser theory states the formalized theorem package, describes the required infrastructure, and then explains the formalization challenges and workflow [1]. The present note follows a similar expository pattern, adapted to percolation.

The project had four stages.

1. The informal proof was rewritten as a formalization-ready proof note, with small mathematical gaps and side conditions filled.
2. The proof note was converted into a Lean blueprint: basic definitions, finite probability infrastructure, connectivity lemmas, independence lemmas, Russo’s formula, and the technical lemmas were organized into a dependency graph.
3. A Codex CLI agent was generated from the blueprint. The agent was instructed to work stage by stage, to run Lean after each stage, and to repair local errors without changing the final theorem statement.

4. The agent was run until the formalization completed. Empirically, the AI first created the framework and filled many easy proofs; the last part of the run was dominated by local repair cycles driven by Lean error messages.

2 The formalized percolation theorem

This section records the mathematical theorem package around which the Lean development was organized. The statements are written in ordinary mathematical notation, but each definition was chosen to correspond closely to a Lean definition.

2.1 Percolation basics

Fix $d \geq 2$. The vertex set is $\mathbb{Z}^d$. Two vertices $x, y \in \mathbb{Z}^d$ are adjacent, written $x \sim y$, if

$$\|x - y\|_1 = 1.$$

The nearest-neighbor bond set consists of unordered adjacent pairs. A percolation configuration assigns to each bond a Boolean state, open or closed. Under $\mathbb{P}_p$, the bond states are independent Bernoulli random variables with open probability p .

For $S \subseteq \mathbb{Z}^d$, we write

$$x \overset{S}{\longleftrightarrow} y$$

if there is an open path from x to y all of whose vertices lie in S . If $S = \mathbb{Z}^d$, we write simply $x \longleftrightarrow y$. We use the ℓ^1 boxes

$$\Lambda_n := \{x \in \mathbb{Z}^d : \|x\|_1 \leq n\}.$$

The infinite connection event is represented through finite events by

$$\{0 \longleftrightarrow \infty\} := \bigcap_{n \geq 1} \{0 \longleftrightarrow \Lambda_n^c\}.$$

Thus

$$\theta(p) := \mathbb{P}_p(0 \longleftrightarrow \infty), \quad p_c := \inf\{p \in [0, 1] : \theta(p) > 0\}.$$

Theorem 2.1 (Sharpness on $\mathbb{Z}^d$). *For nearest-neighbor Bernoulli bond percolation on $\mathbb{Z}^d$, $d \geq 2$, the following hold.*

- (i) *If $p < p_c$, then there exists $c = c(p) > 0$ such that, for every $n \geq 0$,*

$$\mathbb{P}_p(0 \longleftrightarrow \Lambda_n^c) \leq e^{-cn}.$$

- (ii) *If $p_c < p < 1$, then*

$$\theta(p) \geq \frac{p - p_c}{p(1 - p_c)}.$$

The endpoint $p = 1$ is immediate from $\theta(1) = 1$.

2.2 The finite-set criterion

For a finite set $S \subset \mathbb{Z}^d$ with $0 \in S$, define the oriented edge boundary

$$\partial_E S := \{(x, y) : x \in S, y \notin S, x \sim y\},$$

where the edge is oriented from S to its complement. Define

$$\varphi_p(S) := p \sum_{(x,y) \in \partial_E S} \mathbb{P}_p(0 \xrightarrow{S} x).$$

The auxiliary critical point is

$$\tilde{p}_c := \sup \left\{ p \in [0, 1] : \exists S \subseteq \mathbb{Z}^d, 0 \in S, \varphi_p(S) < 1 \right\}.$$

The proof first establishes sharpness with $\tilde{p}_c$ in place of p_c , and then proves $\tilde{p}_c = p_c$.

Two elementary consequences of the definition are formalized explicitly:

$$\begin{aligned} p < \tilde{p}_c &\implies \exists S \subseteq \mathbb{Z}^d, 0 \in S, \varphi_p(S) < 1, \\ p > \tilde{p}_c &\implies \forall S \subseteq \mathbb{Z}^d, 0 \in S, \varphi_p(S) \geq 1. \end{aligned}$$

The first implication uses the supremum definition together with monotonicity of $p \mapsto \varphi_p(S)$.

2.3 Technical lemmas

The Lean development separates the final theorem from the technical lemmas. The following statements form the main proof spine.

Lemma 2.2 (Russo formula, closed-pivotal form). *Let A be an increasing event depending on finitely many edges. For an edge e , say that e is closed-pivotal for A if e is closed, A does not occur, and forcing e to be open makes A occur. Then, for $0 < p < 1$,*

$$\frac{d}{dp} \mathbb{P}_p(A) = \frac{1}{1-p} \sum_e \mathbb{P}_p(e \text{ is closed-pivotal for } A),$$

where the sum is over the finite edge support of A .

Lemma 2.3 (Fundamental differential inequality). *Let $\Lambda \subset \mathbb{Z}^d$ be finite with $0 \in \Lambda$, and set*

$$A_\Lambda := \{0 \longleftrightarrow \Lambda^c\}, \quad f_\Lambda(p) := \mathbb{P}_p(A_\Lambda).$$

Then, for $0 < p < 1$,

$$f'_\Lambda(p) \geq \frac{1}{p(1-p)} \inf_{\substack{S \subseteq \Lambda \\ 0 \in S}} \varphi_p(S) (1 - f_\Lambda(p)).$$

The proof introduces the random set

$$\mathcal{S} := \{x \in \Lambda : x \not\longleftrightarrow \Lambda^c\}.$$

On the event A_Λ^c , the origin lies in $\mathcal{S}$. The closed-pivotal event is decomposed according to the value of $\mathcal{S}$. The crucial formal points are: the event A_Λ depends on finitely many edges; the event $\{\mathcal{S} = S\}$ is independent of the internal edges of S ; and the pivotal edge is naturally oriented from S to S^c .

Lemma 2.4 (Lower bound above $\tilde{p}_c$). *If $\tilde{p}_c < p < 1$, then*

$$\theta(p) \geq \frac{p - \tilde{p}_c}{p(1 - \tilde{p}_c)}.$$

This follows by integrating the differential inequality. The formal proof records the positivity of $1 - f_\Lambda(q)$ before applying logarithms, integrates

$$\int_q^p \frac{dt}{t(1-t)} = \log \frac{p(1-q)}{q(1-p)},$$

and then takes the finite-box limit.

Lemma 2.5 (Boundary inequality). *Let $S \subset \mathbb{Z}^d$ be finite, let $u \in S$, and let $B \subset \mathbb{Z}^d$ satisfy $B \cap S = \emptyset$. Then*

$$\mathbb{P}_p(u \longleftrightarrow B) \leq \sum_{(x,y) \in \partial_E S} p \mathbb{P}_p(u \xrightarrow{S} x) \mathbb{P}_p(y \longleftrightarrow B).$$

Here $y \longleftrightarrow B$ means that y is connected to at least one vertex of B , with the convention that the event holds if $y \in B$.

The proof conditions on the open cluster of u inside S and then uses a last-exit edge from that cluster. The formal independence argument is reduced to disjointness of edge supports: the event determining the internal cluster, the boundary edge xy , and the connection from y to B outside the internal cluster use disjoint sets of edges.

Lemma 2.6 (Exponential decay below $\tilde{p}_c$). *If $p < \tilde{p}_c$, then there exists $c = c(p) > 0$ such that, for every $n \geq 0$,*

$$\mathbb{P}_p(0 \longleftrightarrow \Lambda_n^c) \leq e^{-cn}.$$

Choose a finite $S \ni 0$ with $\varphi_p(S) < 1$, and let L be such that $S \subseteq \Lambda_{L-1}$. If

$$a_n := \mathbb{P}_p(0 \longleftrightarrow \Lambda_n^c),$$

then the boundary inequality gives

$$a_n \leq \varphi_p(S) a_{n-L}, \quad n \geq L.$$

The geometric input is the translation estimate: if $y \in \Lambda_L$ and $y \longleftrightarrow \Lambda_n^c$, then after translating by $-y$ one obtains $0 \longleftrightarrow \Lambda_{n-L}^c$. Iterating the recurrence yields exponential decay.

Proposition 2.7 (Identification of the critical point). *One has $\tilde{p}_c = p_c$.*

The supercritical lower bound gives $p_c \leq \tilde{p}_c$ when $\tilde{p}_c < 1$, and the endpoint case $\tilde{p}_c = 1$ is treated separately. The subcritical exponential bound gives $\theta(p) = 0$ for every $p < \tilde{p}_c$, hence $\tilde{p}_c \leq p_c$.

3 Lean file structure

The Lean repository was organized so that the mathematical dependencies were mirrored by the file dependencies. The exact filenames are not mathematically important, but the following layout captures the successful structure.

```

Sharpness/
  Basic.lean
  Geometry.lean
  Connectivity.lean
  FiniteProbability.lean
  LocalEvents.lean
  Independence.lean
  Russo.lean
  PhiCriterion.lean
  DifferentialInequality.lean
  BoundaryInequality.lean
  Supercritical.lean
  Subcritical.lean
  MainTheorem.lean

```

3.1 Foundational files

The first group of files builds the objects that are not specific to the sharpness theorem.

File	Role
<code>Basic.lean</code>	Global notation, dimension parameter, vertices of $\mathbb{Z}^d$, elementary finite-set conventions, and namespace setup.
<code>Geometry.lean</code>	Nearest-neighbor adjacency, ℓ^1 boxes, boundary vertices and boundary edges, translations, and elementary box inclusions.
<code>Connectivity.lean</code>	Paths, open paths, restricted connectivity $x \xleftrightarrow{S} y$, path concatenation, path restriction, and last-exit lemmas.
<code>FiniteProbability.lean</code>	Finite Bernoulli product measures, cylinder events, monotone events, finite sums, union bounds, and elementary probability identities.
<code>LocalEvents.lean</code>	Events with explicit finite edge support and lemmas saying that connectivity inside a finite set depends only on the corresponding finite edge set.
<code>Independence.lean</code>	Independence of local events with disjoint supports, product-measure factorization, and reusable support-disjointness lemmas.

3.2 Proof-spine files

The second group of files follows the mathematical proof.

File	Main declarations
<code>Russo.lean</code>	Pivotality, closed-pivotality, finite Russo formula, and the homogeneous derivative identity.
<code>PhiCriterion.lean</code>	Definition of $\varphi_p(S)$, definition of $\tilde{p}_c$, monotonicity of $\varphi_p(S)$, and the two consequences of the supremum definition.
<code>DifferentialInequality.lean</code>	Finite dependence of A_Λ , the random set $\mathcal{S}$, measurability of $\{\mathcal{S} = S\}$ with respect to non-internal edges, and the fundamental differential inequality.
<code>BoundaryInequality.lean</code>	Internal clusters $C_S(u)$, last-exit decomposition, support-disjointness for the three events in the boundary expansion, and the boundary inequality.
<code>Supercritical.lean</code>	Integration of the differential inequality, logarithmic side conditions, finite-box limit, and the lower bound above $\tilde{p}_c$.
<code>Subcritical.lean</code>	Choice of S with $\varphi_p(S) < 1$, recurrence $a_n \leq \rho a_{n-L}$, translation estimate, iteration, and construction of the exponential constant.
<code>MainTheorem.lean</code>	Proof of $\tilde{p}_c = p_c$ and final rewriting of the supercritical and subcritical estimates with p_c in place of $\tilde{p}_c$.

3.3 Dependency graph

Figure 1 shows the file-level dependency graph. The purpose of this graph is proof engineering: it tells the agent which facts must exist before a later file can be attempted.

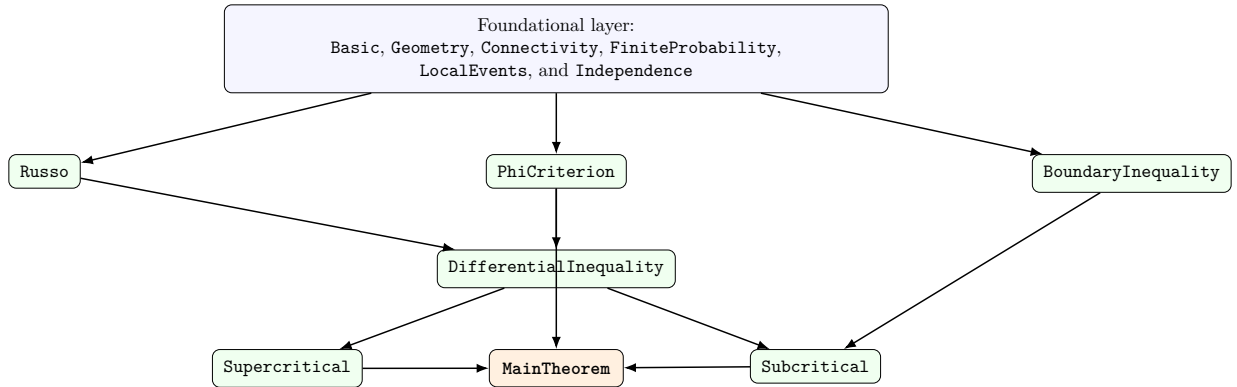


Figure 1: Coarse file-level dependency graph. The foundational layer supplies geometry, connectivity, local events, finite product probability, and independence. The proof spine then follows the mathematical argument: Russo and the finite-set criterion imply the differential inequality; the differential inequality gives the supercritical estimate; the boundary inequality gives subcritical decay; and the two estimates identify the critical point and prove the final theorem.

3.4 Milestone structure

The Codex agent used milestones rather than a single global prompt.

Stage	Target	Acceptance criterion
M0	Skeleton	Files, imports, namespaces, and theorem statements compile; hard proofs may be stubbed locally.
M1	Finite probability	Product-measure API and local-event support lemmas compile.
M2	Geometry and connectivity	Box, boundary, path, last-exit, and translation lemmas compile.
M3	Russo formula	Closed-pivotal Russo formula compiles in the finite setting.
M4	Differential inequality	Lemma 2 compiles from Russo, φ_p , and support-independence facts.
M5	Supercritical and boundary estimates	Lemmas 3 and 4 compile, including logarithmic side conditions and last-exit independence.
M6	Subcritical decay and final theorem	Lemma 5, $\tilde{p}_c = p_c$, and the final sharpness theorem compile with no placeholder on the main theorem path.

4 AI-assisted workflow

This section summarizes the actual workflow. The essential point is that the AI system was not used as a one-shot theorem prover. It was used as a staged proof-engineering assistant, with Lean providing the trusted feedback after every edit.

4.1 Step 1: make the proof formalization-ready

The first prompt asked AI to inspect the mathematical proof, identify hidden gaps, and repair small gaps directly in the proof note. The most useful instruction was:

Rewrite the proof so that every finite-dependence, measurability, independence, monotonicity, endpoint, and limiting argument is explicit. Do not change the theorem statement.

This produced a proof note in which the hard formalization obligations were already named. In particular, the note explicitly isolated the finite dependence of A_Λ , the last-exit characterization of $\{\mathcal{S} = S\}$, the independence argument in the boundary inequality, and the endpoint case $\tilde{p}_c = 1$.

4.2 Step 2: ask for a Lean blueprint

The second prompt asked for a Lean plan rather than code:

Give a Lean formalization plan for this proof. Split the work into modules. Include the basic percolation probability definitions, connectivity definitions, geometric arguments, independence arguments, Russo formula, the technical lemmas, and the dependency relation between every result.

This converted the mathematical proof into a fact graph. The plan emphasized a finite-first strategy: define finite configurations and local events first, prove finite Russo and finite independence statements, and only then pass to infinite percolation through finite boxes and monotone limits.

4.3 Step 3: generate a Codex CLI agent

The third prompt asked AI to generate an agent that would implement the blueprint. The agent contained:

- a controller script, `lean_sharpness_agent.py`;
- an `AGENTS.md` file specifying coding rules;
- milestone prompts M0–M6;
- the proof note and blueprint as references;
- commands for invoking Codex CLI;
- commands for running Lean and writing reports.

A typical stage command was:

```
python3 lean_sharpness_agent.py run \  
--repo ~/Desktop/Leanproject \  
--stage M4 \  
--model gpt-5.5 \  
--iterations 4
```

The full run can be executed by running the milestones sequentially:

```
stages=(M0 M1 M2 M3 M4 M5 M6)  
iters=(1 3 3 4 4 4 4)  
  
for i in "${!stages[@]}"; do  
    extra=""  
    if [ "${stages[$i]}" = "M6" ]; then  
        extra="--require-no-sorry"  
    fi  
    python3 lean_sharpness_agent.py run \  
        --repo ~/Desktop/Leanproject \  
        --stage "${stages[$i]}" \  
        --model gpt-5.5 \  
        --iterations "${iters[$i]}" \  
        $extra  
done
```

The final flag is important: it prevents the agent from declaring success while a placeholder remains on the main theorem path.

4.4 Step 4: run, check, and repair

The successful run had a consistent empirical pattern.

Framework construction. In the first passes, AI created a usable project structure: files, namespaces, imports, definitions, theorem statements, and many placeholders. This stage did not finish the proof, but it fixed the interface between the modules. That was already valuable, because subsequent Lean errors referred to a stable collection of names and statements.

Routine proof filling. Once the framework existed, AI filled many elementary proofs: finite-set inclusions, simple box estimates, path concatenation lemmas, monotonicity of connection events, union bounds, and algebraic simplifications in the supercritical calculation.

Local repair loops. The last stage took the most time. The agent repeatedly ran Lean, read the error messages, and repaired local issues. Typical repairs included adding explicit coercions between sets and finite sets, orienting boundary edges correctly, proving support-disjointness before applying independence, splitting path lemmas into endpoint and vertex-set components, and adding denominator-positivity hypotheses before division or logarithms.

Observed issue	Typical repair
Finite-set coercions	Add explicit membership lemmas and avoid relying on coercion inference.
Path endpoint mismatch	Prove small lemmas for reversing, concatenating, and restricting paths.
Boundary orientation mismatch	State boundary facts using oriented pairs $(x, y) \in \partial_E S$ rather than unoriented edges.
Independence failure	Prove that the finite supports of the events are disjoint before invoking product-measure independence.
Locality failure	Attach an explicit finite support to the event and prove that the event is unchanged outside that support.
Algebraic side condition missing	Add hypotheses such as $0 < p$, $p < 1$, and $1 - f_\Lambda(p) > 0$ before using division or logarithms.
Statement drift	Restore the proof-note statement and adapt helper lemmas to it, instead of weakening the final theorem.

4.5 What made the workflow robust

The main reason the workflow succeeded was that the hard mathematical structure was fixed before code generation began. The AI did not have to discover the Duminil-Copin–Tassion argument while also satisfying Lean. Instead, it had a proof note, a dependency graph, and a staged file structure. Lean then supplied the trusted local feedback.

The resulting loop was:

$$\text{proof note} \longrightarrow \text{Lean blueprint} \longrightarrow \text{Codex edit} \longrightarrow \text{Lean check} \longrightarrow \text{local repair}.$$

This is the main transferable lesson of the project. AI can be very effective at scaffold generation, routine proof search, and local proof repair, but only if the theorem statements and proof architecture are protected from drift.

5 Conclusion

This project gives an example of a successful AI-assisted Lean formalization of a theorem from probability and statistical mechanics. The mathematical proof is short enough to be manageable, but rich enough to test several important formalization mechanisms: finite product probability, graph connectivity, local events, independence, Russo differentiation, recurrence estimates, and passage from finite boxes to an infinite graph.

The workflow also suggests a general template for future projects. First, produce a formalization-ready proof note. Second, convert it into a dependency-aware Lean blueprint. Third, generate a staged agent with narrow milestones. Fourth, use Lean errors as the driver of local repair. In this experiment, the AI system first built the architecture and filled simple proofs, and then spent many cycles closing local gaps. That behavior should be expected in serious formalization projects: the last mile is not a failure of the method, but the part where the proof assistant forces all implicit mathematical bookkeeping to become explicit.

A Representative prompts

Proof-note prompt

Check the mathematics of this proof. Identify any logical gaps or errors. Patch small gaps directly, and report any large errors. Rewrite the proof in a form suitable for Lean formalization, making finite-dependence, independence, measurability, monotonicity, and endpoint arguments explicit.

Blueprint prompt

Give a Lean formalization plan for the proof. Divide the work into modules. Include finite-box percolation definitions, connectivity definitions, geometric arguments, independence arguments, Russo’s formula, the technical lemmas, and a fact graph recording dependencies.

Agent prompt

Build a Codex CLI agent that implements this Lean blueprint stage by stage. The agent should run Lean after each stage, read errors, perform local repairs, and write a report. The final theorem statement must remain the sharpness theorem; changing it counts as failure.

Local repair prompt

Only repair the current Lean errors in this module. Do not rename public declarations and do not weaken theorem statements. Prefer adding small helper lemmas over rewriting the file. Run Lean again and report the remaining errors.

References

- [1] Scott Armstrong and Julia Kempe. Formalization of De Giorgi–Nash–Moser Theory in Lean. *arXiv:2604.05984*, 2026.
- [2] Hugo Duminil-Copin and Vincent Tassion. A new proof of the sharpness of the phase transition for Bernoulli percolation and the Ising model. *Communications in Mathematical Physics*, 343:725–745, 2016.
- [3] Geoffrey Grimmett. *Percolation*. Springer, second edition, 1999.

- [4] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, Lecture Notes in Computer Science 12699, pages 625–635. Springer, 2021.
- [5] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381, 2020.